Web Security Study Guide (W01–W05)

Topics: Web Security Basics (W01), CSRF (W02), XSS (W03), SQL Injection (W04), Clickjacking (W05).

One idea links all five topics: **something is trusted when it should not be.**

Topic	What is wrongly trusted
CSRF	A request that carries my cookie must come from me
XSS	Text from a user is only data, so it is safe to put in a page
SQLi	Text from a user is only data, so it is safe to put in a query
Clickjacking	What I see on screen is what I am clicking

1. Web Security Basics (W01)

1.1 HTTP messages and state

HTTP is stateless at the protocol level. Each request stands alone, and the protocol has no memory of earlier ones: "Web server is stateless".

Applications add state on top. The server keeps session data. The browser keeps a cookie that holds the session ID and sends it back with each request: session data is typically kept on the server. The session starts after login. The ID is a random number, and whoever has the ID has access.

How a message is laid out:

```
REQUEST                                      RESPONSE
+-----------------------------------------+   +-----------------------------------------------+
| POST /post_form.php HTTP/1.1  <- request |   | HTTP/1.1 200 OK               <- status line|
|                                  line   |   |                                               |
| Host: www.example.com         <- headers |   | Content-Type: text/html       <- headers    |
| Cookie: SID=xsdfgergbghedvrbeadv         |   | Set-Cookie: cookieA=aaaaaa                  |
| Content-Length: 19                       |   | Set-Cookie: cookieB=bbbbbb; expires=...;    |
|                                <- blank  |   |             Max-Age=3600                    |
|                                   line   |   |                                <- blank line |
| foo=hello&bar=world            <- body   |   | <h2>Cookies are set</h2>      <- body       |
+-----------------------------------------+   +-----------------------------------------------+
GET puts the data in the URL instead:  GET /post_form.php?foo=hello&bar=world HTTP/1.1
```

- GET and POST differ mainly in how they send data to the server: GET in the URL, POST in the body.
- The status line gives the result: `200 OK` on success, `404 Not Found` when the page does not exist.
- `Set-Cookie` goes from server to browser. PHP `setcookie()` produces it.
- `Cookie` goes from browser to server. The browser attaches all the cookies that belong to the target server, on every later request to it.
- A cookie with no expiry is a session cookie and is lost when the browser closes. A cookie with `expires` or `Max-Age` persists.

```
Browser                                  Server
  | --- POST /login (user, password) ---->  |
  | <--- 200 OK, Set-Cookie: sid=...        |   server stores: sid -> alice
  | --- GET /inbox, Cookie: sid=...  ---->  |   "this sid is Alice"
  | <--- Alice's inbox                      |
```

1.2 First-party and third-party cookies

What the slides say:

- The lecture shows tracking with a 1x1 image from an ad network.
- The lecture says first-party cookies are needed for sites to work, and third-party cookies are used for ads and tracking.
- The lecture names two privacy tools: anonymous (private) mode and blocking third-party cookies.

The definitions below are:

- **Top-level site:** the site shown in the address bar.
- **First-party cookie:** a cookie that belongs to the top-level site.
- **Third-party cookie:** a cookie that belongs to another site, sent when the page loads a resource from that site (image, script, iframe). An ad network's cookie arrives with its 1x1 image on many sites. That lets it link your visits together.
- The same cookie can be first-party on one visit and third-party on another. It depends on which site is at the top.

Same-site is not the same as same-origin:

Pair	Same origin?	Same site?
`https://a.example.com` and `https://a.example.com:443/x`	Yes	Yes
`https://a.example.com` and `https://b.example.com`	No (host)	Yes (same registrable domain `example.com`)
`https://example.com` and `https://example.org`	No	No

SameSite cookie rules and CSRF use **site**. SOP uses **origin**. The lecture defines a same-site request as one from a page of the site itself, and a cross-site request as one from a page of another site.

1.3 Cookie flags

Each flag deals with a different threat. They do not replace each other, and none is complete on its own.

Flag	Threat it deals with	What it does	What it does not do
`HttpOnly`	Cookie theft by injected script	Script cannot read the cookie through `document.cookie`	Does not stop the browser sending it. Does not stop CSRF. Does not stop injected script from making requests that carry it
`Secure`	Sniffing on the network	Sent only over HTTPS	Does not hide the cookie from script. Does not stop CSRF
`SameSite=Strict`	CSRF	Not sent on any cross-site request	Even a link click from another site arrives without it, so the user looks logged out
`SameSite=Lax`	CSRF	Sent on cross-site top-level GET navigation (a link click). Not sent on cross-site POST, img, iframe or Ajax	Does not protect a state-changing GET behind a link
`SameSite=None`	—	Sent on all requests (needs `Secure`)	No CSRF protection

The slides name SameSite as a CSRF defence. The full Strict/Lax/None rules are. See 2.6 for how the slide simplifies Lax.

1.4 Same-Origin Policy (SOP) and CORS

Origin = scheme + host + port. All three must match.

Page compared with `http://a.com/x`	Same origin?
`http://a.com/y`	Yes (path is not part of origin)
`https://a.com/x`	No (scheme)
`http://b.a.com/x`	No (host)
`http://a.com:8080/x`	No (port)

What SOP does to Ajax:

- Ajax means Asynchronous JavaScript and XML. The slide code uses `XMLHttpRequest`. Ajax is slowly being replaced by the Fetch API.
- The demo uses two sites, `www.bank32.com` and `www.bank99.com`. One slide title describes a bank32 page reaching bank99. The packet capture shows the reverse case: a page on bank99 (`ajax.html`) sends an Ajax GET to `www.bank32.com/getdata.php`. The rule is the same either way.
- The capture shows the request goes out, with an `Origin: http://www.bank99.com` header, and the server answers `200 OK` with "Data from Bank32!".
- The browser then refuses to give the response to the script. The console shows "Cross-Origin Request Blocked ... CORS header 'Access-Control-Allow-Origin' missing".
- The reason is privacy. Ajax code in a Facebook page may read the user's Facebook data, but not the user's Google data.

So: **a page can send a cross-origin request, but its script cannot read the response.** This is the key fact behind CSRF. The attacker can send but cannot read.

CORS (Cross-Origin Resource Sharing) relaxes this, because SOP is sometimes too strict. The server gives the browser a list of origins it trusts. In the slide, the PHP page on `www.bank99.com` sends `header("Access-Control-Allow-Origin: http://www.bank32.com")` with its reply ("Time from Bank99"). Pages from bank32 can then read bank99's response; pages from other origins still cannot.

The JavaScript sandbox. Script from a web page runs inside a sandbox in the browser. The lecture lists what must be protected: page data, the local file system, and the network. Each has its own rule:

- **Page data and DOM**: script can read and change its own page through the DOM.
- **File system**: script cannot read local files directly. The user grants access by picking a file in a file chooser, and the script then gets a handle to that file only.
- **Network**: normal HTTP, Ajax and WebSocket each have a different policy (SOP above, WebSocket in 1.5).

1.5 Ajax vs WebSocket

The lecture lists three ways a page talks to a server: normal HTTP requests, Ajax and WebSocket. Each has its own policy.

	Ajax (XMLHttpRequest, now often Fetch)	WebSocket
Protocol	HTTP	Starts as an HTTP request, then upgrades to a long-lived connection
Direction	Half-duplex: the browser asks, the server answers. The server cannot push data on its own	Full-duplex: either side sends at any time, with no request needed
Cross-origin connection	Request is sent	**Connection attempt is allowed**
Cross-origin data	Browser blocks reading unless CORS allows it	**Browser does not restrict the data**
Who enforces access	The client side: the browser, using the server's CORS list	**Only the server**, by checking the `Origin` header in the handshake

How the WebSocket handshake works:

1. The page opens `ws://` or `wss://` to any host.

2. The browser sends an HTTP GET with `Upgrade: websocket` and an `Origin` header naming the page's origin. Script cannot fake this header.
3. The browser also attaches that host's cookies.
4. If the server accepts, both sides can send messages, and the page reads replies freely.

If the server does not check `Origin`, any web page you visit can talk to it as you. That includes servers on your home network, which are normally out of reach.

Cable Haunt (January 2020):

```
  Internet                          |   Home network (LAN)
                                    |
[Attacker web page] ---- loads ---> [Victim browser]
  script inside it                  |      |
                                    |      | 1. script opens WebSocket to the modem's LAN address
                                    |      |    (browser allows the connection; no SOP block)
                                    |      v
                                    |  [Broadcom-based cable modem]
                                    |   WebSocket server, did not check Origin
                                    |      | 2. crafted message overflows a buffer
                                    |      v
                                    |   attacker code runs on the modem
                      trust boundary-+ (the modem trusted anything from inside the LAN)
```

The browser is the bridge. It sits inside the LAN, and it runs code from the internet.

1.6 How a web app is built

The lecture has three parts: the web architecture, the web server, and HTTP with cookies, then JavaScript and its sandbox.

- **HTML** (Hypertext Markup Language) builds the page.
- **CSS** (Cascading Style Sheets) sets the look, so content stays apart from presentation.
- **JavaScript**, also called ECMAScript, is the scripting language of web pages. It can be included inline, as an event attribute, or as a linked file (see 3.3, how the browser parses it).
- **Dynamic content** once also used Adobe Flash, Microsoft Silverlight, ActiveX and Java applets.
- The **HTTP server** (for example Apache) handles requests and files. The **web application server** runs the code that builds pages. They connect by CGI (a new process per request), FastCGI (a faster form of CGI) or server modules that run the scripts directly.
- The architecture: the browser talks to a web server over HTTP; the web server runs the application, which may use a database.
- Apache **virtual hosting**: one server hosts many sites and picks one by the `Host` header (`ServerName`). All the labs use this.
- PHP can be written inline inside HTML, or kept apart from HTML using templates.
- GET puts parameters in the URL (`?name=value&name2=value2`), and POST puts them in the body. A GET should not change state, but many apps do it anyway.

1.7 Trust boundaries

```
[Attacker] -- hosts a page or plants input -->  [Victim browser] -- holds cookies for the site
                                                        |  browser attaches cookies by itself
                                                        v
                                          [Web server] -- builds a query --> [MySQL]
                                           trusts cookie                      trusts the query text
```

1.8 Quick revision (W01)

1. HTTP is stateless. Apps keep state with server-side sessions and a cookie that holds the session ID.
2. Origin = scheme + host + port. Site = registrable domain. They are different tests.
3. SOP: the request goes out, but the response cannot be read.
4. CORS is a relaxation of SOP that the server approves.
5. WebSocket: the browser allows the cross-origin connection and does not restrict its data. The server must check `Origin`.
6. `HttpOnly` stops reading the cookie. `Secure` stops sending it over plain HTTP. `SameSite` stops sending it cross-site. Each flag covers a different threat.

Practice questions: Web basics

1.1 A page at `https://shop.com/cart` sends an Ajax GET to `https://shop.com:8443/api/items`. Can its script read the response?
Answer: No, unless the server allows it with CORS. The port differs, so the origin differs. SOP lets the request go out but blocks reading the response.

1.2 An attacker's page opens a WebSocket to a server the victim can reach. What stops it?
Answer: Only the server, by checking the `Origin` header in the handshake. Browsers allow cross-origin WebSocket connections and do not restrict their data.

1.3 A session cookie has `HttpOnly` and `Secure`. What can still go wrong?
Answer: CSRF. The browser still attaches the cookie to a forged cross-site request. The two flags only stop script reading it and stop it being sent over plain HTTP.

1.4 You visit `news.com`. It loads an image from `ads.net`, and the request carries an `ads.net` cookie. First-party or third-party?
Answer: Third-party. What counts is the top-level site in the address bar, not who set the cookie. The same cookie is first-party when you visit `ads.net` directly.

2. CSRF — Cross-Site Request Forgery (W02)

2.1 Core idea

Same-site and cross-site requests. A request from a page back to its own website is a same-site request. A request to a different website is a cross-site request. Example: a page that is not Facebook holds a Facebook link; clicking it sends a cross-site request to Facebook.

You are logged in to a site. The browser **attaches your cookie to every request to that site, no matter which page started it**. This is called **ambient authority**. If another page makes your browser send a request to that site, the site sees a valid cookie and believes you meant it.

Using the cookie alone, the server **cannot tell a same-site request from a cross-site one**, so a third-party site can forge a request that looks exactly like a same-site one. The browser knows the difference, but the server does not.

What an attack needs: a target website; a victim with an active session on it; and a website the attacker controls. The attacker builds a page that sends the forged request, and gets the logged-in victim to visit it.

2.2 Step by step

```
1. Victim logs in to www.seed-server.com      -> browser stores the session cookie
2. Victim, in another tab, opens a page on www.attacker32.com
3. That page makes the browser send a request to www.seed-server.com
4. Browser adds the victim's cookie by itself
5. Server sees a valid cookie and performs the action
6. The attacker never sees the response (SOP). It does not need to.
```

2.3 Two ways to force a request

GET puts data such as `foo` and `bar` in the URL; POST puts it in the body.

- **GET:**
- Bank example: a logged-in user at `www.bank32.com` transfers money with a URL like `transfer.php?to=3220&amount=500`. The forged request must leave from the victim's own browser, so that the browser adds the victim's session cookie.
- Put the target URL in anything that loads a URL, such as an `img` or `iframe` tag. The browser fetches the `src` address and the action runs. This works only because the app changes state on GET.
- In the Elgg demo the image is made very small so the victim does not notice. The page sits on the attacker's site in `/var/www/CSRF/Attacker`. Samy sends Alice the link in a private message; when she opens it, Samy is added to her friend list.
- **POST:**
- A normal HTML form sends a POST when the user clicks Submit. The attacker's job is to "click" it without the user.
- The attacker's script: (1) creates a form with method POST, (2) adds the fields as hidden inputs and puts the form in the page, (3) submits it, and (4) runs this function automatically when the page loads. The user sees nothing.

POST timeline:

```
time   Victim browser                   attacker32.com          seed-server.com
 |     logged in, has session cookie
 |---- GET attacker page --------------->|
 |<--- page: hidden form + auto-submit --|
 |     (page loads, script runs)
 |---- POST /action/profile/edit ------------------------------->|
 |     Cookie: session (added by browser)                        |
 |     body: guid, profile fields, access level                  |  cookie valid,
 |                                                               |  no token check
 |<--- response (attacker's page cannot read it) ----------------|  -> profile changed
 v
```

Both kinds of attack need the exact request format. The attacker learns it by doing the action on its own account and watching the request.

- The browser's developer tools (Network tab) show each request's URL, headers and body.

2.4 What a hands-on CSRF exercise shows

Practice labs often use Elgg, an open-source social network, with its token check turned off.

- A captured request (add friend, edit profile) has three parts: the action URL with its values, the token fields, and the session cookie that the browser adds itself. With the token check off, everything except the cookie is predictable.
- A GET action can be forged with only an image tag. Modern browsers may not send cross-site cookies on such requests. That is the browser protecting the user.
- A POST action needs a hidden form that submits itself. Values such as the victim's user ID must be found in public pages first, so one forged page targets one known victim. XSS reads such values at run time instead (3.4).
- With the token check back on, the forged request fails. The attacker's page cannot read the token, because of SOP.

2.5 Defences

The lecture sorts the defences by who helps:

- **The browser helps:** the Referer header and same-site cookies.
- **The server helps itself:** a secret token.

Defence	How it works	Weakness
Referer / Origin check	The server looks at where the request came from	Referer reveals browsing history, so privacy tools and proxies often remove it. The server must then either accept requests that lack it (attacks pass) or reject them (real users break).

Defence	How it works	Weakness
SameSite cookie	The browser does not attach the cookie on cross-site requests	Needs browser support. Lax still sends it on cross-site GET navigation
Secret token	Each page carries a random value that the attacker cannot know. The server rejects requests that lack the right one	Strong only if built and checked correctly (see below)

Why the token stops CSRF:

- The token is random and differs per user.
- The attacker's page cannot read the real site's pages, because SOP blocks it.
- So the attacker can only send a request without the token, and the server rejects it.

What can still break a token:

- **XSS on the same site.** Injected script runs in the site's origin and can read the token (section 3).
- **Token leakage**, for example in a URL that ends up in logs or in the Referer header.
- **Weak checks**, for example:
- accepting a request when the token is missing,
- checking only that a token exists, not that it is right,
- checking only some methods or actions.
- **No session binding.** A token not tied to the user's session can be swapped for the attacker's own valid token.
- **Predictable tokens** that the attacker can guess.

How Elgg's token works:

- Each request carries two values: `__elgg_ts` (a timestamp) and `__elgg_token`.
- The page holds them in JavaScript variables (`elgg.security.token.__elgg_ts` and `elgg.security.token.__elgg_token`) and in hidden form fields.
- The two fields are `__elgg_ts` (timestamp) and `__elgg_token`.
- The token is built from four inputs: the site secret, the timestamp, the user's session ID and a random session string.
- It is an HMAC (keyed hash) built on MD5.

2.6 SameSite in detail

The lecture says SameSite is supported "in browsers like Chrome and Opera". It lists **Strict (not sent with cross-site requests)** and **Lax (sent with cross-site requests)**. The Lax line is a simplification. Lax sends the cookie on cross-site **top-level GET navigation** only. It does not send it on cross-site POST, image, iframe or Ajax requests.

SameSite experiment:

1. Visit `http://www.example32.com/`. It sets three cookies: `cookie-normal`, `cookie-lax` and `cookie-strict`.
2. **Link A** opens a page on `www.example32.com` (same-site). **Link B** opens a page on `www.attacker32.com` (cross-site). The two pages differ only in background colour.
3. Each page sends requests to `www.example32.com/showcookies.php`, which prints the cookies it received. The request types are a link (GET), a GET form and a POST form.

Expected results under current browser rules:

Request	Link A (same-site)	Link B (cross-site)
Link click (GET navigation)	normal, lax, strict	normal, lax
GET form submit (navigation)	normal, lax, strict	normal, lax
POST form submit	normal, lax, strict	normal only

Some browsers treat a cookie with no SameSite attribute as Lax. Then even `cookie-normal` is missing from the cross-site POST.

The repo's CSRF lab has a README note on SameSite but no `www.example32.com` test site. Its `apache_csp.conf` is the XSS CSP set-up (3.7). To run the experiment, use the official SEED image.

2.7 Common mistakes

- CSRF does **not** steal the response. It makes the victim *do* something.
- Cookies are sent by the browser, so `HttpOnly` does not help.
- POST is not safer than GET against CSRF. The attacker only needs a form.
- The attacker never sees the victim's password or cookie value, and does not need them.
- SOP keeps the token secret from other sites. Same-site script (XSS) is not blocked.

2.8 Quick revision (CSRF)

1. Cause: ambient authority (cookies are sent automatically).
2. Needs: the victim is logged in, the request format is predictable, and the victim visits the attacker's page.
3. GET through tags, POST through a hidden form that submits itself.
4. Defences: Referer (weak, often missing), SameSite (browser), secret token (strong if done right).
5. A token fails to XSS, to leakage, and to weak or unbound checks.

Practice questions: CSRF

2.1 A bank uses only a session cookie. Why can a page on another site cause a transfer?
Answer: Ambient authority. The browser attaches the bank's cookie to any request sent to the bank, whatever page started it. The

attacker never needs the cookie's value.

2.2 Why does a secret token stop CSRF but not XSS?
Answer: SOP stops the attacker's cross-site page from reading the token. Injected script runs in the site's own origin, so it can read the token from the page.

2.3 What is wrong with defending only by checking the Referer header?
Answer: Referer can be missing, because privacy tools and proxies remove it. The server must then accept such requests (attacks pass) or reject them (real users break).

2.4 With `SameSite=Lax`, which cross-site request still carries the cookie?
Answer: Top-level GET navigation, such as a link click. Cross-site POST, image, iframe and Ajax requests do not carry it.

3. XSS — Cross-Site Scripting (W03)

3.1 Core idea

The site puts user-supplied text into a page without making it safe. The browser cannot tell that the site did not write this text, so it runs it as script **with the site's own origin and privileges**. The injected code can do whatever the user can do: read cookies, change the page and send requests.

Root cause: **code and data are mixed.**

3.2 Types

The lecture names two types: **non-persistent (reflected)** and **persistent (stored)**. DOM-based XSS is not in the slides.

Type	Where the bad input lives	Who is hit
Stored (persistent)	Saved on the server: a profile field, comment or message	Everyone who later views it
Reflected (non-persistent)	Sent in the request (URL or form), echoed back in the response, not saved. Example: a search page that prints the `input` parameter	The person who follows the crafted link
DOM-based	The page's own JavaScript reads input (for example the URL fragment after `#`) and writes it into the page with an unsafe sink such as `innerHTML`	The person whose browser runs that page script

```
STORED                              REFLECTED                          DOM-BASED
attacker -> POST profile text       attacker sends a crafted link       attacker sends a crafted link
            |                                 |                                  |
         [server DB] saves it          victim clicks -> request to server   victim opens the page
            |                                 |                                  |
victim -> GET profile page          server copies input into response   server returns a normal page
            |                                 |                                  (may never see the input)
server sends page with the text     browser parses response                        |
            |                                 |                                 page JS reads location/hash
browser parses -> script runs       script runs in site origin                     and writes it into the DOM
in site origin                                                                              |
                                                                                   script runs in site origin
```

More from the slides:

- **Reflected**: many sites take input, do something with it, and put the original input back into the response page. The slides call this "reflective behaviour". If the input holds script, the site itself delivers that script to the page.
- **Stored**: when a site stores one user's data and later shows it to others, it creates a channel between users. A social-network profile is one: one user sets it, another views it. These channels are meant to carry data only. If the site does not clean the input, markup and script travel through the channel and run in the viewer's browser with the site's privileges.
- **Fix for stored and reflected:** encode on the server when building HTML.
- **Fix for DOM-based:** use safe sinks in the client code, such as `textContent` instead of `innerHTML`, plus CSP. Encoding on the server does not help when the server never handles the input.

The lab covers stored XSS only (the Elgg profile).

3.3 How the browser parses it

```
Server response (HTML text)
        |
   HTML parser reads tags
        |  sees a script element (from the site OR from user text - it cannot tell)
        v
   JavaScript engine runs it in the page's origin
        -> can read the DOM, the page's JS variables, send requests as the user
```

Code gets into a page in two ways:

- **Inline:** a `<script>` block, or an event attribute such as `onclick`.
- **Linked:** `<script src="...">`, pointing to a file on the same site or another site.

Inline is the problem, because the browser cannot tell who wrote it. A linked file at least has a known source, which CSP can check.

3.4 What the attacker can do

Three kinds of damage: defacing the page, forging requests as the user, and stealing information.

What a hands-on XSS exercise shows (script stored in a social-network profile field):

- Any field shown to other users is an attack surface. Rich-text editors wrap input in extra HTML, so tests use the editor's plain HTML mode.

- The steps build up: prove script runs, show it can read the page's cookie, show it can send data to another server, then show it can act as the user (add a friend, edit the profile) with Ajax.
- Sending data out works because the browser makes any request the script asks for, including one to the attacker's server.
- Acting as the user works even with CSRF tokens on. The page keeps its token in JavaScript variables, the script reads it, and its request then matches a real one.
- The script reads values such as the victim's user ID at run time. A CSRF page must know them in advance.
- The injected script comes from the site itself, so it may read the site's cookies. A CSRF page is third-party and cannot.
- `HttpOnly` stops only the cookie-reading steps. Acting as the user still works, because the browser attaches the cookie to the script's requests.
- A copy that spreads itself becomes a worm (3.5). The best-known case is the Samy worm on MySpace (4 October 2005), which reached over one million users in about 20 hours.

3.5 Worm: how it copies itself (idea only)

Two approaches:

- **Link approach**: the worm is an external file (`xssworm.js`). The infected profile holds only a script tag pointing to it. To spread, it copies that one tag. This is easy, but it needs a server to host the file.
- **DOM approach**: the whole worm sits inline in the page. To spread, it rebuilds a copy of itself: 1. Get its own element, `document.getElementById("worm")`, and read its `innerHTML`. This gives the code inside the element without the `<script>` tags. 2. Add the opening and closing tags back. The closing tag is written in two pieces, so the HTML parser does not end the script block early. 3. URL-encode the result with `encodeURIComponent`, because the POST body type is `application/x-www-form-urlencoded`. 4. Post it into the victim's profile. The lab requires this approach, because it needs no outside server.

3.6 Defences

Defence	Idea	Notes
Filtering	Remove dangerous tags from input	Hard, because there are many ways to embed code besides a script tag. The slides' Java example is jsoup. Elgg uses the HTMLawed plugin, a highly customizable PHP sanitizer. The lab turns it off by commenting out its hook call in `filter_tags()` in `input.php`
Encoding	Turn special characters into harmless text (`<` becomes `<`), so they are shown, not run	Elgg uses PHP `htmlspecialchars()`. The lab comments it out in the output views `dropdown.php`, `text.php` and `url.php`
CSP (Content Security Policy)	The server tells the browser which script sources are allowed. Inline script and inline event handlers are blocked unless allowed	Set as an HTTP header by Apache or by the app

CSP in the slides:

- **The problem**: inline code. The fix is to forbid inline code and allow only linked code from trusted places.
- **Origin allow-list**: `Content-Security-Policy: script-src 'self' example.com https://apis.google.com`. Scripts may load from the page's own origin and from those two hosts only.
- **Nonce**: `script-src 'nonce-34fo3er92d'`. An inline script with `nonce=34fo3er92d` runs, and one with `nonce=3efsdfsdff` does not. The server must create a fresh unguessable nonce for each response.
- **Hash**: allows an inline script whose content hash matches a hash listed in the policy. A changed script no longer matches, so it is blocked.
- **Setting it**: in PHP, `header("Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-1rA2345' www.example.com")`, sent before the page body.

3.7 CSP worked example

One test page has six script areas and a button. Three sites serve it with different policies, and two more sites host script files.

Site	CSP set by	Policy
`www.example32a.com`	nothing	none
`www.example32b.com`	Apache `Header set`	`default-src 'self'; script-src 'self' *.example70.com`
`www.example32c.com`	PHP (`phpindex.php`)	`default-src 'self'; script-src 'self' 'nonce-111-111-111' *.example70.com`
`www.example60.com`, `www.example70.com`	nothing	host JavaScript files

What each area shows:

Area	Code	32a (no CSP)	32b (Apache CSP)	32c (PHP CSP)
1	inline, nonce 111-111-111	OK	Failed	OK
2	inline, nonce 222-222-222	OK	Failed	Failed
3	inline, no nonce	OK	Failed	Failed
4	from self	OK	OK	OK
5	from example60.com	OK	Failed	Failed
6	from example70.com	OK	OK	OK

Area	Code	32a (no CSP)	32b (Apache CSP)	32c (PHP CSP)
button	inline `onclick`	works	blocked	blocked

This table is worked out from the policies and the page, not from a browser run.

Why:

- `'self'` allows files from the same origin.
- A nonce allows only inline script that carries the matching nonce.
- `*.example70.com` allows that host.
- Anything not allowed is blocked.
- Inline event handlers are blocked unless `'unsafe-inline'` is allowed.

What it shows:

- With no CSP, everything runs. That is why injected inline code is dangerous.
- To allow scripts from another host, add the host to `script-src`. Adding `*.example60.com` makes area 5 run.
- To allow one inline block, add its nonce. Adding `'nonce-222-222-222'` makes area 2 run.
- Inline event handlers stay blocked under both policies. Move them into a script file and attach them with `addEventListener`.
- CSP helps against XSS because the browser runs only code from sources the server named. Injected inline code does not run.

3.8 Common mistakes

- Stored vs reflected: stored input lives on the server; reflected input comes back in the same request.
- XSS runs in the **site's** origin, so SOP does not stop it. That is the whole point.
- `HttpOnly` does not stop XSS. It only stops reading the cookie.
- A CSRF token stops CSRF, not XSS.
- Encoding output is the main fix. Filtering alone is easy to get wrong.
- CSP is a second layer, not a replacement for encoding.

3.9 Quick revision (XSS)

1. Cause: untrusted data is treated as code.
2. Types: stored and reflected (slides), DOM-based (general).
3. Damage: deface the page, forge requests, steal data.
4. XSS beats CSRF tokens because the script reads the token. `HttpOnly` blocks only cookie reading.
5. Worm: link approach vs DOM approach (the lab requires DOM).
6. Fixes: filter, encode, CSP (origin allow-list, nonce, hash).

Key comparisons:

- CSRF and XSS both say "cross-site", but CSRF sends a request from another site and cannot read the target's pages. XSS runs code inside the target's own pages, so it can read and do anything the user can.
- CSRF defences do not stop XSS. The script reads the token from the page, and its requests are same-site, so SameSite cookies are sent too.

Practice questions: XSS

3.1 A comment is saved and shown to every visitor without encoding, and script in it runs. Which type is this, and why is it powerful?
Answer: Stored XSS. It runs in the site's own origin, so SOP does not restrict it.

3.2 A site has `HttpOnly` cookies and CSRF tokens, but a stored XSS bug remains. What can injected script still do?
Answer: Read the token from the page and send valid requests as the user. `HttpOnly` only stops it reading the cookie.

3.3 Policy: `default-src 'self'; script-src 'self' 'nonce-abc'`. Which runs: inline script with no nonce, an inline `onclick`, inline script with `nonce="abc"`, or a script from `http://evil.com/x.js`?
Answer: Only the inline script with `nonce="abc"`. Inline code needs the matching nonce, inline handlers need `'unsafe-inline'`, and `evil.com` is not in the allow-list.

3.4 What is the best long-term fix for XSS?
Answer: Encode user data when writing it into HTML, so the browser treats it as text, not markup. Removing words like "script" is easy to get around, and `HttpOnly` or POST do not stop script from running.

4. SQL Injection (W04)

4.1 SQL basics

- Log in to MySQL, then `CREATE DATABASE`, `SHOW DATABASES` and `USE db`. SQL keywords are not case-sensitive.
- `CREATE TABLE` and `DESCRIBE`. The slide example is table `employee` in database `dbtest`, with seven columns. The notes cover data types, `NOT NULL` and auto-increment.
- `INSERT INTO` adds a row, and the ID column fills itself.
- `SELECT` reads all columns or chosen ones. The slide example reads EID5001, David.
- `WHERE` filters rows. Conditions combine with `AND` and `OR`. A condition that is always true (`1=1`) selects every row. It looks useless in a normal query but matters in injection.
- `UPDATE ... SET a=..., b=... WHERE ...` changes rows. The SET list is separated by commas.
- Three comment styles: `#` to end of line, `--` to end of line, and `/* ... */`. In MySQL, `--` must be followed by a space or control character; the slide does not say this.

4.2 How a web app reaches the database

```
User -> browser -> web server (PHP) -> MySQL
  form data in a GET/POST request
  PHP puts it in $_GET or $_POST
  PHP connects: new mysqli("localhost", user, password, db)
  PHP builds the query STRING with the data inside
  PHP sends the string to MySQL
```

- Three parts: the browser, the web application server and the database. The browser talks to the server in HTTP. The server talks to the database in SQL.
- An HTML form sends its fields as a GET request by default. PHP receives them in the `$_GET` or `$_POST` array.
- `new mysqli(...)` takes four arguments: host, login name, password and database name. The host is `"localhost"` when the database runs on the same machine.
- PHP has three ways to reach MySQL: the old MySQL extension, MySQLi (common, MySQL 4.1+) and PDO.
- HTTPS encrypts the request on the network. It does nothing about what the data does inside the query.
- The path from user input to the database is a new attack surface: if the data is not handled with care, it can change the query.

The weak point: **the user's data becomes part of the SQL text.** The database cannot tell which part the developer wrote and which part the user typed.

4.3 The core mechanism

1. The developer means for the user to fill in a blank inside quotes.
2. The attacker types a quote, which closes the string early.
3. The attacker's next text is then read as SQL.
4. A comment symbol cuts off the rest of the original query, such as the password test.

So the query now means something the developer did not plan.

Concatenation vs binding, as query trees:

```
CONCATENATED  input for eid = EID5002'#

  text sent:  SELECT * FROM employee WHERE eid='EID5002'#' AND password='...'
                                                  ^ rest is a comment
  parsed as:
        SELECT
          └─ WHERE
               └─ eid = 'EID5002'           <- password test is gone

BOUND  same input, prepared statement

  template:   SELECT * FROM employee WHERE eid=? AND password=?   (parsed first)
  parsed as:
        SELECT
          └─ WHERE
               └─ AND
                    ├─ eid = [value 1]       <- value 1 = the text  EID5002'#  (no row has it)
                    └─ password = [value 2]
  The tree is fixed before the values arrive. The values never change its shape.
```

Results from the slides:

- `EID5002'#` with any password returns that employee's name, salary and SSN.
- An always-true condition returns every record.
- With `curl`, special characters must be URL-encoded: `%27` for an apostrophe, `%20` for a space and `%23` for `#`. Put the whole URL in single quotes so the shell does not treat `&` as a command separator.
- The UPDATE page (`changepasswd.php`, a POST with EID, old password and new password) lets Alice (EID5000) add `salary=...` to the SET list and raise her own salary. The slide asks how to set Bob's (EID5001) salary to 0. For that, the WHERE part must also be changed to point at Bob's row.

4.4 Worked example: login and UPDATE

- **Login.** The code reads the username and password, hashes the password, and builds a SELECT with both values in quotes, run with `$conn->query()`. Only the username reaches the SQL as raw text, so it is the injection point. If the matched row is the admin, the page lists every employee.
- The same request works from the web form or from the command line with curl, as long as special characters are URL-encoded.
- A second statement after a semicolon fails, because `query()` runs one statement only (4.5).
- **Profile edit.** The code runs one `UPDATE credential SET ... WHERE ID=$id` built from user values. Injected text can add one more `column=value` to the SET list (such as salary) or change the WHERE target to another user's row.
- Passwords are stored as hashes. Changing a password through injection means storing the hash of a known password, not the plain text.

4.5 Multiple statements

Adding a second statement after a semicolon, such as a drop-database command, does **not** work here. PHP `mysqli::query()` refuses to run more than one statement, so the slide's attempt to add a drop of `dbtest` fails. The page then shows a MySQL syntax error for the extra text. `mysqli::multi_query()` can run several statements, but it is **not recommended**.

So the damage is limited to changing the one existing statement, unless the app uses `multi_query()`.

4.6 Fundamental cause

The cause is mixing data and code. The slide shows this as a diagram, with a note to see the textbook for detail. The same cause sits behind SQL injection, XSS, attacks on the `system()` call, and format-string bugs. The matching fix for `system()` is `execve()`, which takes the command and its data as separate arguments.

4.7 Defences

1. Filtering and encoding. Escape special characters before they join the query. In PHP this is `mysqli::real_escape_string()`, which encodes characters such as quotes, NULL, `\r`, `\n` and `\b`.

- The slide notes say this does not fix the fundamental cause, because data and code are still mixed. It only makes the code safer.
- It only helps when the value is inside quotes. A numeric field with no quotes stays open. It also depends on the right character-set set-up.

2. Prepared statements (the fundamental fix).

The lecture calls them a **performance feature** for running the same statement many times. They were not built for security:

> "The database parses, compiles and performs query optimization on the SQL statement template and stores the result without executing it."

```
Step 1  prepare:  send the TEMPLATE with ? placeholders   -> DB parses, compiles, optimises, stores it. Nothing runs.
Step 2  bind:     send the VALUES (bind_param, "is" = int, string)
Step 3  execute:  run the stored statement with the bound values
```

Why it is safe:

- The template goes through the **code channel** and the values go through the **data channel**.
- "Data received from the data channel is not parsed."
- So SQL-looking text in a value stays plain data.

General principle:

- The statement's structure is fixed apart from the values, and the values are treated only as data.
- Some drivers do not send a real prepare to the server. PHP PDO with emulated prepares (the default for MySQL) escapes and inserts the values on the client.
- That is still safe for values when the character set is set correctly. But the "server compiles first" picture is not literally what happens there.

A safe login query with mysqli:

```
$stmt = $conn->prepare("SELECT name, local, gender FROM USER_TABLE WHERE id = ? and password = ?");  // prepare
$stmt->bind_param("is", $id, $pwd);   // bind; i = integer, s = string
$stmt->execute();                     // execute and fetch
$stmt->bind_result($bind_name, $bind_local, $bind_gender);
$stmt->fetch();
```

Limit: placeholders can stand in for **values** only. They cannot replace table names, column names or `ORDER BY` choices. For those, use a fixed allow-list. So "100% secure" is an overclaim: prepared statements stop injection through values, not every SQL mistake.

4.8 Testing tip

The web app hides error messages. Copy the final SQL string into the MySQL console to find syntax errors.

4.9 Common mistakes

- In MySQL, `--` needs a space after it; `#` does not.
- `query()` blocks stacked statements; `multi_query()` allows them.
- If the password is hashed before it enters the query, only the **username** is the injection point. Without hashing, both fields are.
- Prepared statements protect values, not identifiers.
- `real_escape_string` is not the same as prepared statements.
- UPDATE injection changes data; SELECT injection reads data.

4.10 Quick revision (SQLi)

1. Cause: user data joins the query text (code and data are mixed).
2. A quote closes the string, and a comment cuts off the rest.
3. SELECT = read; UPDATE/INSERT = change.
4. `query()` runs one statement; `multi_query()` runs many.
5. Fixes: escaping (weaker), prepared statements (strong).
6. Prepare, then bind, then execute. Code channel and data channel.

Practice questions: SQL injection

4.1 Login code puts the username inside quotes and hashes the password with SHA1 before building the query. Which field is the injection point?
Answer: The username. The password becomes a hex hash with no quotes in it before it enters the SQL text. Without the hashing, both fields would be open.

4.2 Input holds a semicolon and a second statement. The app uses `mysqli::query()`. What happens?
Answer: The call fails. `query()` runs one statement only; `multi_query()` would run both.

4.3 Why can SQL-looking text in a bound value not fool a prepared statement?
Answer: The query's structure is fixed before the values arrive. Bound values are sent as data and never parsed as SQL.

4.4 "Prepared statements make every query injection-proof." Where is this wrong?
Answer: Placeholders cannot stand for identifiers such as column names or an `ORDER BY` choice. Those must be picked from an allow-list, never built from user input.

5. Clickjacking (W05)

5.1 Core idea

The victim thinks they are clicking on one page. They are really clicking on another page hidden on top of it. The target site receives a genuine click from a logged-in user, so nothing looks wrong to the server. The cookies are real; the user's intent is not.

5.2 Building blocks

The deck has four parts: iframes and their properties, clickjacking with iframes, countermeasures (CSP), and the iframe's own security features.

- **iframe:** embeds one web page inside another. The lecture shows pages that hold other pages this way, each with its own `src`, `width` and `height`.
- **Overlapping iframes:** CSS `position: absolute` with `top` and `left` puts one frame over another.
- **Transparent iframe:** `opacity: 0` makes the frame invisible but still clickable.

Layers, as the attacker builds them (CSS values are typical):

```
               z-index   opacity   what it is
viewer's eye
   |
   v   +-------------------------------------------+
       | layer 2: iframe of target site            |  z-index: 2   opacity: 0     receives the click
       |          [ Add friend ]  <- lined up      |  (invisible, but on top)
       +-------------------------------------------+
       +-------------------------------------------+
       | layer 1: attacker's bait page             |  z-index: 1   opacity: 1     what the user sees
       |          "Win a prize!"  [ CLICK HERE ]   |
       +-------------------------------------------+
The user aims at CLICK HERE. The click hits the invisible Add friend button above it.
```

5.3 Attack variants

1. **LikeJacking.** A social "Like" button is placed with absolute positioning (for example `top:220px; left:700px`) exactly where the bait asks the user to click. One click "likes" the attacker's page. The slide's attack page frames a bait page (`bottom1.html`) and puts the transparent Like frame on top.

Each variant has an "Attack Code" slide. They all use the same pieces: an iframe of the target, absolute position, z-index and opacity, as in the diagram above.

2. **Adding a friend.** A hidden frame of the target site's add-friend page is lined up under the bait button. The user adds the attacker as a friend without knowing.
3. **Sequence of clicks.** Some actions need several clicks, for example "Delete", then "Are you sure? Yes". The bait page shows several buttons (btn1–btn4), each placed over the next real button in turn, often as a game. Each click moves the hidden action one step on.
4. **Small-size iframe.** The attacker's page holds a very small frame (for example 120x50 pixels, `scrolling="no"`, no border) positioned at a chosen spot. Because it is tiny and cannot scroll, it shows only one small part of the other page, such as a button or a few words. That part blends into the host page and looks like the attacker's own content. The slide example shows one site's text inside another site's page.
5. **Fake login.** The attacker's own fake login form is placed where the real one appears, so the user types their password into the attacker's form.

5.4 Building and defending a test page

1. **Attacker:** frame the target page full-size with no border, layered under the attacker's content using CSS `position`.
2. **Attacker:** place a decoy button exactly over a real button in the frame, so the click reaches the framed page.
3. **Defender:** add a framebuster. If `window.top` is not `window.self`, move the top window to this page.
4. **Attacker:** add `sandbox` to the iframe. Top navigation is blocked, so the buster cannot escape and the page stays framed.
5. **Defender:** send `X-Frame-Options: DENY` and `Content-Security-Policy: frame-ancestors 'none'`. The browser refuses to show the page in any frame, and no script on either side can change that.

5.5 Defences

Frame-busting (framekiller). Script on the page checks whether it is inside a frame and, if so, moves the top window to itself:

```
if (top != self) top.location.replace(location);
```

The slides call it unreliable and no longer used. The reason is that the attacker controls the frame (5.6).

How `sandbox` defeats the buster. Two things are different: whether scripts **run**, and whether scripts may **navigate the top window**.

Attacker's iframe	Buster script runs?	Can it move the top window?	Result
no `sandbox`	yes	yes	buster escapes; attack fails
`sandbox` (no options)	**no**, scripts are off	—	buster never runs; page stays framed
`sandbox="allow-scripts"`	**yes**, `top != self` is true	**no**, `allow-top-navigation` is missing, so the browser blocks the navigation	buster runs but cannot escape; page stays framed

Attacker's iframe	Buster script runs?	Can it move the top window?	Result
`sandbox="allow-scripts allow-top-navigation"`	yes	yes	buster escapes

The attacker usually also adds `allow-forms` so the framed page's buttons still submit. The buster is weak because the attacker chooses the sandbox options, not the target.

X-Frame-Options (XFO). A response header, set in PHP with `header("X-Frame-Options: DENY")`:

- `DENY`: no page may frame this one.
- `SAMEORIGIN`: only pages from the same origin may.
- The slides call it the first fix, now outdated.

Results from the slides' demo (pages on `www.bank32.com`):

Framed from	No XFO	`DENY`	`SAMEORIGIN`
`www.attacker32.com`	shown	blocked	blocked
`www.bank32.com` (same origin)	shown	blocked	shown

A blocked frame shows Firefox's "Can't Open This Page".

CSP `frame-ancestors`: the modern fix.

- Syntax: `Content-Security-Policy: frame-ancestors <source> ... <source>;`. Sources can be `'none'`, `'self'` or listed origins.
- It covers `frame`, `iframe`, `object`, `embed` and `applet`.
- The demo pages are `csp-all.php`, `csp-self.php` and `csp-url.php`.
- It can be set per page in PHP, or for a whole site in Apache:

```
<VirtualHost *:80>
    ServerName www.bank32.com
    DocumentRoot "/var/www/bank32"
    Header set Content-Security-Policy "frame-ancestors 'self'"
</VirtualHost>
```

In browsers that support both headers, `frame-ancestors` takes priority over XFO. Neither works from a `<meta>` tag; each must be a real HTTP header.

Other CSP directives in the slides: `script-src` and `img-src`.

5.6 iframe security features

Same-Origin Policy applies between frames. The slide experiment:

```
 Parent page P  (www.attacker32.com)          each page holds an element id="secret"
   ├── iframe A (www.attacker32.com/iframe1)
   ├── iframe B (www.attacker32.com/iframe2)
   └── iframe C (www.bank32.com/iframe1)
 Access code: child.contentDocument.getElementById("secret")
              parent.document.getElementById("secret")
              siblingframe.contentDocument.getElementById("secret")
```

Who accesses whom	Result
P, A and B with each other (same origin)	Allowed
P, A or B reading C	Blocked
C reading P, A or B	Blocked

The slides test three directions: parent to child, child to parent, and child to sibling. In class, the discussion slide asks you to predict each result first; the result slide gives the answers, which are the table above.

Different origins can still embed each other, but they cannot read each other.

The `sandbox` attribute, written `<iframe sandbox="<options>" src=...>`:

- Options that turn features back on: `allow-same-origin`, `allow-scripts`, `allow-forms`, `allow-modals` and `allow-top-navigation`.
- With no options, the frame gets a unique origin, and it has no scripts, no forms, no pop-ups and no top navigation.

Use case: show dynamic content from outside, such as user posts, in a sandboxed iframe, so code injected in it cannot harm your page. The lecture builds such content, which includes an alert script, sets `iframe.srcdoc = content; iframe.sandbox = "";`, and the script does not run. This guards against code injection, as CSP does.

Do not give an embedded page both `allow-scripts` and `allow-same-origin` if it comes from your own origin. Its script can then remove the sandbox.

5.7 Common mistakes

- Clickjacking is **not** CSRF. The user really clicks, so tokens do not help.
- Framebusting is weak. The real fix is a **header** (XFO or CSP `frame-ancestors`).
- `frame-ancestors` controls who may frame **me**. `sandbox` controls what **I** allow a frame I embed to do. Do not mix them up.
- `<meta>` tags do not carry XFO or `frame-ancestors`.
- `sandbox` with `allow-scripts` still blocks top navigation unless `allow-top-navigation` is also given.

5.8 Quick revision (Clickjacking)

1. A hidden or transparent iframe sits over a bait page.

2. Variants: LikeJacking, adding a friend, a click sequence, a small iframe that shows one piece of another page, a fake login.
3. Framebusting is unreliable. A sandboxed frame stops it: with no scripts, or with scripts but no top navigation.
4. XFO: `DENY` and `SAMEORIGIN`. CSP: `frame-ancestors`, also settable in Apache.
5. iframe SOP: frames from different origins cannot read each other.
6. `sandbox` options, and the `srcdoc` example for untrusted content.

Practice questions: Clickjacking

5.1 Why can a page with a JavaScript framebuster still be framed?
Answer: The attacker frames it with `sandbox` and without `allow-top-navigation`, so the buster cannot move the top window.

5.2 A site wants only itself and `https://partner.com` to frame its pages. What should it send?
Answer: The HTTP header `Content-Security-Policy: frame-ancestors 'self' https://partner.com`. Modern browsers ignore `ALLOW-FROM`, `SAMEORIGIN` cannot name a partner, and `frame-ancestors` is ignored in a `<meta>` tag.

5.3 Does a CSRF token stop clickjacking on an "Add friend" button?
Answer: No. The real page loads in the frame with a valid token, and the victim's own click submits the real form.

5.4 The attacker frames the target with `sandbox="allow-scripts allow-forms"`. The target has a framebuster. What happens?
Answer: The buster runs, but the browser blocks the top navigation, so the target stays framed.

6. Mixed questions and rapid revision

6.1 Mixed questions

M1. A hidden form on another site posts to a site. Separately, the user clicks a button shown in an invisible frame. Which defences match?
Answer: The first is CSRF: a secret token or SameSite. The second is clickjacking: `frame-ancestors` or `X-Frame-Options`. A token does not stop a real click.

M2. Why can an XSS bug defeat the site's CSRF token, while the attacker's own site cannot?
Answer: XSS script runs in the site's origin, so SOP lets it read the token from the page.

M3. What is the main difference between CSRF and XSS?
Answer: CSRF sends requests from another site and cannot read the replies. XSS runs code inside the target site and can read and act as the user.

M4. Can a secret token or a same-site cookie defend against XSS?
Answer: No. The injected script's requests come from the site itself, so they are same-site and can include the token.

M5. Match `HttpOnly`, `Secure` and `SameSite` to the threat each one deals with.
Answer: `HttpOnly`: script reading the cookie. `Secure`: sending it over plain HTTP. `SameSite`: sending it on cross-site requests (CSRF).

M6. Which attacks share the cause "data and code are mixed"?
Answer: SQL injection, XSS, command injection through `system()`, and format-string bugs. CSRF and clickjacking do not; they abuse cookies and framing.

M7. A page on `evil.com` fetches `bank.com/balance` with Ajax and also opens a WebSocket to `bank.com`. Neither server does anything special. What can its script read?
Answer: Not the Ajax response, because there is no CORS header. WebSocket messages, yes, if the server accepts the connection without checking `Origin`.

6.2 Definitions

- **SOP:** scripts may read only same-origin resources. Origin = scheme + host + port. Requests go out, but responses cannot be read across origins.
- **CSRF:** the attacker makes the victim's browser send a request to a site, with the victim's cookie.
- **XSS:** the attacker's script runs in the victim's browser, in the site's origin.
- **SQLi:** user data changes the meaning of a SQL query.
- **Clickjacking:** the victim's click lands on a hidden element from another site.
- **CSP:** a header that tells the browser which sources of script, images and frames are allowed.
- **Prepared statement:** the statement template is fixed first; values are bound later, through a data channel, and are not parsed as SQL.

6.3 Comparison table

	CSRF	XSS	SQLi	Clickjacking
What it abuses	The user's authority	The user's browser session	The database	The user's click
Needs victim logged in?	Yes	Not always	No	Usually yes
Attacker sees response?	No	Yes (script runs in-origin)	Depends on query	No
Root cause	Cookies sent automatically	Code and data mixed	Code and data mixed	Page can be framed
Main fix	Token / SameSite	Output encoding / CSP	Prepared statements	`frame-ancestors` / XFO
Beaten by	XSS, leaked or weakly checked token	Weak or missing encoding	Identifiers built from input	A framebuster alone (sandbox beats it)

6.4 Attack sequences

- **CSRF:** log in, visit the attacker's page, the browser sends a request with the cookie, the action is done.

- **Stored XSS:** the attacker saves input, the victim loads the page, the script runs in the site's origin, then data is stolen or a request is forged.
- **XSS worm:** view an infected page, the script forges a profile edit, it copies itself into the victim's profile, it spreads.
- **SQLi login:** a quote ends the string, a comment cuts the rest, login without the password.
- **Clickjacking:** the victim opens the attacker's page, a transparent frame sits on top, the real click hits the target button.

6.5 Defence map

Defence	Stops
Secret token	CSRF (not XSS)
SameSite cookie	Most cross-site cookie sending
Referer check	Weak CSRF defence
Output encoding	Stored and reflected XSS
Safe DOM sinks (`textContent`)	DOM-based XSS
HTMLawed / jsoup filtering	Some XSS (fragile)
CSP `script-src` (origin, nonce, hash)	XSS
CSP `frame-ancestors`, XFO	Clickjacking
`sandbox` iframe	Untrusted embedded content
Prepared statement	SQLi through values
`real_escape_string`	Some SQLi (weaker)
`Origin` check on WebSocket handshake	Cross-site WebSocket use

6.6 Common mistakes

1. `HttpOnly`, `Secure` and `SameSite` deal with different threats. None replaces the others.
2. SOP blocks reading responses, not sending requests.
3. Browsers allow cross-origin WebSocket connections and do not restrict their data. The server must check `Origin`.
4. A CSRF token is kept secret by SOP. XSS, leakage and weak or unbound checks can still beat it.
5. Framebusting is weak. Headers are the real fix, and `<meta>` does not carry them.
6. `query()` runs one statement; `multi_query()` runs many.
7. In MySQL, `--` needs a space after it.
8. Prepared statements do not cover identifiers.
9. The Elgg token is called an MD5 digest in the slides and is an HMAC with MD5 in the lab code.
10. The labs turn off Elgg's countermeasures (CSRF token, HTMLawed, `htmlspecialchars`) on purpose.
11. Same-site is not the same as same-origin.
12. A sandboxed frame can run scripts (`allow-scripts`) and still be unable to move the top window.

7. Burp Suite: a tool for testing web apps

Burp Suite, made by PortSwigger, is a set of tools for testing the security of web applications. It sits between the browser and the server, so every request and response from sections 1 to 5 can be seen, paused and changed. Use it only on apps you own, on practice labs, or where you have written permission to test.

7.1 Editions

Edition	Cost	Notable tools
Community	Free	Proxy, Repeater, Intruder (rate-limited), Decoder, Comparer, Sequencer, Burp's browser
Professional	Paid, free trial	Everything in Community, plus Scanner, Collaborator, engagement tools and reports

7.2 How the proxy works

```
[Burp's browser] ---request---> [Burp Proxy] ---request---> [Web server]
                 <--response---  listener      <--response--
                                 127.0.0.1:8080 (default)
                                 Intercept on: hold each message
                                 HTTP history: log every message
```

- **Burp's browser** (Proxy > Intercept > **Open browser**) is a Chromium build that is already set up to use the proxy. Nothing needs configuring.
- **Another browser** must be pointed at the listener (`127.0.0.1:8080` by default). For HTTPS sites it must also trust Burp's CA certificate, which you download from `http://burpsuite` while the proxy is on. Without that, the browser shows certificate errors, because Burp decrypts and re-encrypts the traffic.
- **Intercept on:** each request stops in Burp. You can read it, edit it, then click **Forward** (send it) or **Drop** (discard it).
- **Intercept off:** traffic flows freely, but is still logged in **Proxy > HTTP history**.

7.3 Main tools

Tool	What it does	Where it helps in this guide
Proxy	Captures and edits traffic between browser and server	See cookies, `Set-Cookie` flags, `Origin`, `Referer` (1.1 to 1.5)
Target > Site map	Tree of every page and request seen so far	Map the app before testing

Tool	What it does	Where it helps in this guide
Repeater	Edit one request by hand and resend it as often as you like	Check if a request still works with no CSRF token, or with the `Origin` changed (2.5)
Intruder	Sends many versions of one request, changing marked positions from a list	Test many inputs against one field; guess IDs
Decoder	Encodes and decodes URL, HTML, Base64 and hex	Read URL-encoded bodies and encoded output (3.6, 4.3)
Comparer	Shows the difference between two requests or responses	Compare a normal response with a changed one
Sequencer	Measures how random session IDs or tokens are	Check that a CSRF token or session ID is not predictable (2.5)
Scanner (Pro)	Crawls the app and checks it for known flaws	Automated check for XSS, SQLi and more
DOM Invader	Extension in Burp's browser that traces input into DOM sinks	Find DOM-based XSS (3.2)
Clickbandit	Builds a framed test page for a chosen click	Check if a page can be framed (5.5)
Collaborator (Pro)	A server that records outside lookups an app makes	Find flaws that only show up out-of-band

7.4 A first session

1. Install Burp and start a temporary project with default settings (**Next**, then **Start Burp**).
2. Go to **Proxy > Intercept** and click **Open browser**.
3. Browse the test site with intercept off, so HTTP history fills up.
4. Set the scope: in **Target > Site map**, right-click the host and choose **Add to scope**. Click **Yes** to stop logging out-of-scope traffic. In HTTP history, filter to **Show only in-scope items**. This hides traffic to other sites such as analytics and video embeds.
5. Find a request of interest in HTTP history. Right-click and choose **Send to Repeater**.
6. In Repeater, change one thing at a time, click **Send**, and read the response. The arrows step back through earlier attempts.
7. Turn intercept on only when you need to change a request before it reaches the server.

7.5 What to look for, per topic

- **Cookies (1.3):** does the login response's `Set-Cookie` carry `HttpOnly`, `Secure` and `SameSite`?
- **CORS (1.4):** does the server reflect any `Origin` back in `Access-Control-Allow-Origin`, together with `Access-Control-Allow-Credentials: true`?
- **WebSocket (1.5):** the handshake and messages appear under **Proxy > WebSockets history**. Does the server reject a changed `Origin`?
- **CSRF (2):** send a state-changing request to Repeater, remove the token or change its value. A safe server answers with an error such as `403`.
- **XSS (3):** send a harmless marker such as `<b>test123</b>` and search the response. If it comes back as `<b>test123</b>`, output is encoded.
- **SQLi (4):** a single quote that causes a database error in the response shows input reaches the query unsafely.
- **Clickjacking (5):** does the page's response carry `X-Frame-Options` or `Content-Security-Policy: frame-ancestors`?

7.6 Quick revision

1. Burp is a proxy between browser and server. Default listener `127.0.0.1:8080`.
2. Burp's browser works with no setup. Other browsers need the proxy setting and Burp's CA certificate for HTTPS.
3. Proxy captures, Repeater resends one request by hand, Intruder sends many, Decoder converts encodings, Comparer diffs, Sequencer tests randomness.
4. Scanner and Collaborator are Professional only.
5. Set the target scope first, so the history shows only the site under test.
6. Test only what you are allowed to test.

Practice questions: Burp Suite

7.1 Firefox uses Burp's proxy. HTTP pages work, but every HTTPS page shows a certificate warning. What is the fix?
Answer: Install and trust Burp's CA certificate in Firefox. Burp decrypts HTTPS and re-encrypts it with its own certificate.

7.2 Which tool checks whether a profile-edit request still works with its CSRF token removed?
Answer: Repeater. It lets you edit one request by hand, resend it and read the response.

7.3 Which tool tells you whether session IDs are random enough?
Answer: Sequencer. It collects many tokens and measures how random they are.

7.4 HTTP history is full of requests to analytics and video sites. How do you show only the app under test?
Answer: Add the app's host to the target scope, then filter HTTP history to "Show only in-scope items".

7.5 Name a tool that is only in the Professional edition.
Answer: Scanner (Collaborator too). Proxy, Repeater and Decoder are in both editions.

Further reading: PortSwigger's guide at https://portswigger.net/burp/documentation/desktop/getting-started, and the free Web Security Academy labs.